Математическая постановка и фактическое исполнение GPU path tracer

Ревизия исходного кода проекта `path_trace_golang`

22 октября 2026 г.

Contents

1 Назначение документа и границы точности **2**

1.1 Что было исправлено относительно старой версии 2

2 Общая постановка задачи **2**

3 Фактический pipeline: от вызова Go до PNG **3**

4 Данные и раскладка памяти **3**

4.1 Камера, небо и туман 3

4.2 Материалы и объекты 4

5 Параллелизм и индексы **4**

6 Первичный луч камеры **5**

7 Пересечения и геометрическая нормаль **5**

7.1 Сфера 5

7.2 Плоскость 6

7.3 AABB 6

8 Поверхности и throughput **6**

8.1 Lambert 6

8.2 Metal: GGX visible-normal sampling 6

8.3 Mirror и гладкий metal 7

8.4 Dielectric: интерфейс сред 7

9 Прямое освещение и MIS **7**

9.1 Список света 7

10 Итеративный path loop **8**

11 Объёмный туман **8**

12 Накопление, HDR и постобработка **9**

12.1 Тонемаппинг и 8-bit output 9
12.2 CPU denoise и smooth 10

13 Фактические настройки исполнения **10**

14 Проверка соответствия коду **10**

15 Итоговая математическая схема **11**

1 Назначение документа и границы точности

Этот документ описывает не абстрактный «идеальный» path tracer, а текущую цепочку исполнения проекта: выбор backend в `internal/usecases/render/service.go`, упаковку сцены в `internal/transport/gpu/packing.go`, compute shader из `shaders.go`, вспомогательные shader-модули и обратную передачу результата. Формулы ниже соответствуют реализации на 22.10.2026; приоритет имеют имена функций и фактические операции шейдера.

CPU backend реализует ту же семантику трассировки в `float64`, но имеет собственный worker pool и собственный RNG. Поэтому одинаковый seed делает каждый backend воспроизводимым сам с собой, но не обязан давать попиксельно одинаковую последовательность случайных чисел.

1.1 Что было исправлено относительно старой версии

Старое утверждение	Фактическое исполнение
Work group 16×16	Обычный path shader: 32×32; wireframe: 16×16.
RGBA16F	Текстуры и readback: RGBA32F; накопление: три float32 на пиксель.
Стратификация 4×4	В path shader используется независимое jitter-сэмплирование; батч из 4 sample/pixel не является стратификацией.
Schlick Fresnel	Для диэлектрика используется точная неполяризованная формула Френеля; для металла — Schlick-подобная цветная аппроксимация.
NEE только через $\cos\theta_i \cos\theta_o/(r^2 p_A)$	Сфера сэмплируется по телесному углу, коробка — по видимой площади с преобразованием PDF.
Firefly clamp $L_{\max} = 500$	В текущем GPU path shader такого clamp нет.
Temporal accumulation через предыдущую текстуру	Текстура действительно ping-pong-ится, но shader не читает `prevFrameTex`; математическое накопление выполняет только `accumSSBO`.

2 Общая постановка задачи

Пусть сцена задаётся множеством объектов $\mathcal{O}$, материалов $\mathcal{M}$, источников света $\mathcal{L}$, камерой и средой. Для изображения ширины W и высоты H требуется оценить цвет каждого пикселя (x, y):

$$\widehat{C}(x,y) = \frac{1}{S}\sum_{k=0}^{S-1} L(x,y,k), \qquad S = \max(\texttt{SamplesPerPx}, 1).$$

Здесь L — HDR-линейный RGB-радианс одного пути. Глубина пути ограничена `MaxDepth`=D, а расстояние пересечения — `MaxRayDist`; если оно не положительно, GPU использует $R_{\max} = 1000$.

Поверхностная часть оценивает уравнение рендеринга:

$$L_o(\boldsymbol{x}, \omega_o) = L_e(\boldsymbol{x}, \omega_o) + \int_{\Omega^+} f(\boldsymbol{x}, \omega_i, \omega_o) L_i(\boldsymbol{x}, \omega_i)(\boldsymbol{n} \cdot \omega_i)\, d\omega_i.$$

В объёме добавляется затухание и single-scattering через коэффициенты σ_s, σ_a, $\sigma_t = \sigma_s + \sigma_a$.

3 Фактический pipeline: от вызова Go до PNG

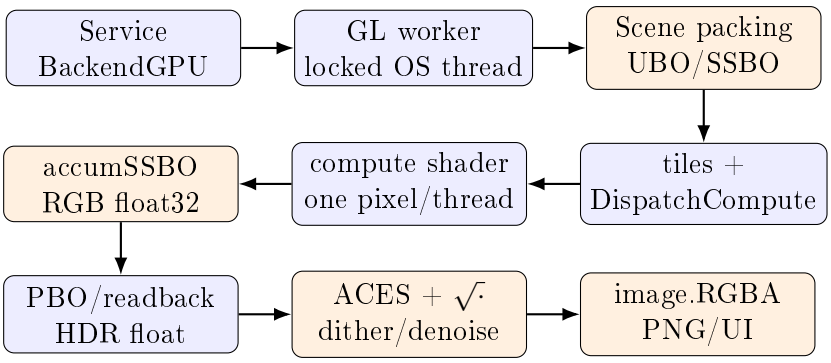


Figure 1: Реальная последовательность GPU backend.

1. `Service.IntoWithCancel` выбирает GPU только при `BackendGPU`; при ошибке инициализации/рендера выполняется CPU fallback.
2. `RenderWithCancel` создаёт скрытое GLFW-окно, запускает один GL worker и привязывает контекст к его OS thread. Нужен OpenGL 4.3 и compute shader.
3. `prepareGPUFrame` упаковывает сцену, изменяет размеры ресурсов, очищает texture и накопительный SSBO, загружает uniform/SSBO и выбирает normal/wireframe.
4. Один проход обрабатывает не весь кадр одним вызовом: изображение режется на адаптивные плитки. Каждая плитка ждёт fence, чтобы не переполнять очередь GPU и разрешать UI обрабатывать отмену.
5. После dispatch выполняется barrier, затем при live preview делается периодический readback через один из двух PBO. После последнего pass гарантируется финальный readback.
6. HDR RGB преобразуется в 8-bit RGBA, затем при подходящих настройках идут CPU bilateral denoise и необязательный box smooth.

4 Данные и раскладка памяти

4.1 Камера, небо и туман

Все три UBO передаются как массивы по 16 float32, то есть по 64 байта:

$$C = [p_x, p_y, p_z, 0,\ t_x, t_y, t_z, 0,\ u_x, u_y, u_z, 0, \mathrm{FOV}, a, f, A],$$

где a — aperture, f — focus distance, A — aspect ratio.

$$Sky = [s, 0, 0, 0,\ c_r, c_g, c_b, 1,\ h_r, h_g, h_b, 1,\ z_r, z_g, z_b, 1].$$

Значение $s = 0$ означает deprecated background, $s = 1$ — solid, $s = 2$ — gradient. Для fog:

$$F = [d, q, a_s, v,\ c_r, c_g, c_b, 1, \sigma_s, \sigma_a, g, k, n, o, 0, 0],$$

где d — legacy density, q — scatter, a_s — affect sky, v — GPU volumetric, k — heterogeneity strength, n — noise scale, o — number of octaves.

4.2 Материалы и объекты

Для материала i используется ровно 20 float32 (80 байт):

$$[type, rough, ior, smoothness,\ albedo_{rgb}, reflectivity,\ emit_{rgb}, pad, absorption_{rgb}, absorptionScale, tint_{rgb}, pad].$$

При упаковке emission уже умножается на $\max(0, \texttt{Power})$. Для dielectric при нулевых legacy-полях задаются IOR 1.5, tint $(1,1,1)$ и absorption scale 0.01; для metal/mirror при нулевых полях smoothness и reflectivity становятся соответственно $1 - rough$ и 1.

Для объекта используется 12 float32 (48 байт):

$$[type, materialIndex, 0, 0,\ position_{xyz}, 0,\ size_{xyz}, 0].$$

Sphere использует $size_x$ как радиус, plane имеет нормаль $(0,1,0)$, box задаётся

$$\boldsymbol{b}_{min} = \boldsymbol{p} - \frac{1}{2}\boldsymbol{s}, \qquad \boldsymbol{b}_{max} = \boldsymbol{p} + \frac{1}{2}\boldsymbol{s}.$$

Индекс источника хранится отдельно как int32 в `lightIndices`; туда попадают только emissive-материалы с хотя бы одной положительной компонентой emission.

Накопитель имеет $3WH$ float32:

$$\text{base} = 3(yW + x), \qquad A_{\text{base:base+3}} \leftarrow A + L.$$

RGBA32F texture и PBO имеют четыре float32 на пиксель, но четвёртая компонента служит только альфой при записи $(R, G, B, 1)$; в математическое накопление она не входит.

5 Параллелизм и индексы

Для обычного shader объявлено $l_x = l_y = 32$, поэтому одна work group содержит 1024 invocations. Для wireframe используется 16×16. Для участка плитки размером $w \times h$:

$$G_x = \left\lceil \frac{w}{l_x} \right\rceil = \frac{w + l_x - 1}{l_x}, \qquad G_y = \left\lceil \frac{h}{l_y} \right\rceil.$$

Глобальная координата потока с offset (o_x, o_y):

$$x = \texttt{gl_GlobalInvocationID.x} + o_x, \qquad y = \texttt{gl_GlobalInvocationID.y} + o_y.$$

Потоки за границей кадра сразу выходят по условию $x \geq W \vee y \geq H$.

Обычный path pass обрабатывает B сэмплов на пиксель, где по умолчанию $B = 4$ и допустимы $1 \ldots 16$. Число pass:

$$P = \left\lceil \frac{S}{B} \right\rceil, \qquad B_i = \min(B, S - iB), \qquad N_i = iB + B_i.$$

В shader каждый из B_i sample использует индекс $k = N_i - B_i + j$. Начальный RNG:

$$q = yW + x, \qquad state_0 = \text{hash}(q \,\text{xor}\, \text{hash}(frameSeed + k)).$$

Затем каждый вызов `rng` делает

$$state \leftarrow hash(state + \texttt{0x9e3779b9}), \qquad \xi = \frac{state \gg 8}{2^{24}} \in [0, 1).$$

Здесь hash — последовательность XOR, сдвигов и умножений на `0xed5ad4bb`, `0xac4c1b51`, `0x31848bab`.

6 Первичный луч камеры

Пусть $\boldsymbol{c}$ — position, $\boldsymbol{t}$ — target, $\boldsymbol{u}_0$ — up:

$$\boldsymbol{w} = \frac{\boldsymbol{c} - \boldsymbol{t}}{\|\boldsymbol{c} - \boldsymbol{t}\|}, \qquad \boldsymbol{u} = \frac{\boldsymbol{u}_0 \times \boldsymbol{w}}{\|\boldsymbol{u}_0 \times \boldsymbol{w}\|}, \qquad \boldsymbol{v} = \boldsymbol{w} \times \boldsymbol{u}.$$

Для $\theta = \mathrm{FOV}\,\pi/180$:

$$h = \tan(\theta/2), \quad V_h = 2h, \quad V_w = AV_h, \quad H = fV_h\boldsymbol{v}, \quad W_c = fV_w\boldsymbol{u},$$

$$\boldsymbol{q} = \boldsymbol{c} - \tfrac{1}{2}W_c - \tfrac{1}{2}H - f\boldsymbol{w}.$$

Jitter выбирается без стратификации:

$$\xi_x, \xi_y \sim U[0,1), \qquad U = \frac{x + \xi_x}{W}, \qquad V = \frac{H - 1 - y + \xi_y}{H}.$$

Направление pinhole-луча:

$$\boldsymbol{d} = \mathrm{normalize}(\boldsymbol{q} + UW_c + VH - \boldsymbol{c}).$$

При aperture > 0 выбирается равномерная точка на диске объектива:

$$r_l = \frac{aperture}{2}\sqrt{\xi_1}, \quad \phi = 2\pi\xi_2, \quad \boldsymbol{o} = \boldsymbol{c} + \boldsymbol{u}(r_l\cos\phi) + \boldsymbol{v}(r_l\sin\phi),$$

а направление строится из $\boldsymbol{q} + UW_c + VH - \boldsymbol{o}$. Это даёт DOF с фокусом на плоскости, заданной f.

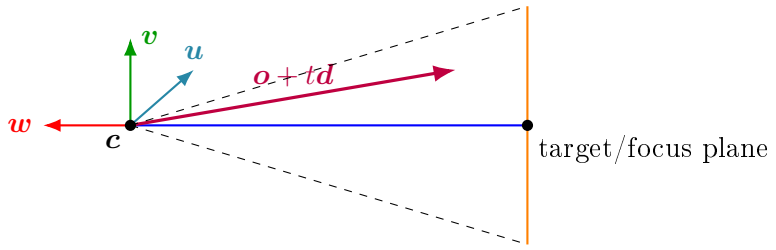


Figure 2: Построение луча из камеры; координата y переворачивается перед вычислением V.

7 Пересечения и геометрическая нормаль

Все пересечения ищутся в интервале $[t_{min}, t_{max}]$; для продолжения пути `hitWorld` перебирает объекты в порядке массива и оставляет ближайшее пересечение, каждый раз уменьшая t_{max}.

7.1 Сфера

Луч $\boldsymbol{r}(t) = \boldsymbol{o} + t\boldsymbol{d}$, сфера $(\boldsymbol{c}, r)$:

$$a = \boldsymbol{d}\cdot\boldsymbol{d}, \quad b_h = (\boldsymbol{o} - \boldsymbol{c})\cdot\boldsymbol{d}, \quad c = (\boldsymbol{o} - \boldsymbol{c})^2 - r^2, \quad \Delta = b_h^2 - ac.$$

При $\Delta \geq 0$ проверяются корни

$$t = \frac{-b_h \pm \sqrt{\Delta}}{a}.$$

В текущем normal shader используется эквивалентная midpoint-форма:

$$t_m = -\frac{(\boldsymbol{o} - \boldsymbol{c})\cdot\boldsymbol{d}}{a}, \qquad \boldsymbol{q} = (\boldsymbol{o} - \boldsymbol{c}) + t_m\boldsymbol{d}, \qquad h = \sqrt{\frac{r^2 - \boldsymbol{q}^2}{a}}, \quad t = t_m - h.$$

Если первый корень вне интервала, берётся $t_m + h$. Внешняя нормаль $\boldsymbol{n}_o = \mathrm{normalize}(\boldsymbol{r}(t) - \boldsymbol{c})$.

7.2 Плоскость

Для точки $\boldsymbol{p}$ и нормали $\boldsymbol{n} = (0, 1, 0)$:

$$t = \frac{(\boldsymbol{p} - \boldsymbol{o}) \cdot \boldsymbol{n}}{\boldsymbol{d} \cdot \boldsymbol{n}}.$$

При нулевом знаменателе или $t \notin [t_{min}, t_{max}]$ пересечения нет.

7.3 AABB

Для каждой оси i:

$$t_0^i = (b_{min}^i - o^i)/d^i, \quad t_1^i = (b_{max}^i - o^i)/d^i, \quad t_{near} = \max_i \min(t_0^i, t_1^i), \quad t_{far} = \min_i \max(t_0^i, t_1^i).$$

Пересечение существует при $t_{far} \geq t_{near}$; если вход уже позади t_{min}, берётся t_{far}. Для продолжения диэлектрического пути есть отдельный режим `findExit`, который принудительно берёт выходную грань. Нормаль грани получается из оси, обновившей t_{near} или t_{far}, а не из сглаживания.

После пересечения применяется `setFaceNormal`:

$$frontFace = [\boldsymbol{d} \cdot \boldsymbol{n}_o < 0], \qquad \boldsymbol{n} = frontFace ? \boldsymbol{n}_o : -\boldsymbol{n}_o.$$

Вторичный луч сдвигается от поверхности через ULP-based `offsetRay`; это подавляет self-intersection и не является физическим расстоянием в сцене.

+

8 Поверхности и throughput

Для пути хранятся $\boldsymbol{\beta}$ (throughput), $\boldsymbol{L}$ (накопленная radiance), текущая среда, pdf_{prev}, предыдущая точка и η-масштаб. Начальные значения:

$$\boldsymbol{\beta} = (1, 1, 1), \qquad \boldsymbol{L} = (0, 0, 0), \qquad etaScale = 1.$$

8.1 Lambert

$$f_r = \frac{\boldsymbol{\rho}}{\pi}, \qquad p(\omega) = \frac{\boldsymbol{n} \cdot \omega}{\pi}, \qquad \boldsymbol{\beta} \leftarrow \boldsymbol{\beta} \odot \boldsymbol{\rho}.$$

Направление генерируется cosine-weighted sampling:

$$\phi = 2\pi\xi_1, \quad \cos\theta = \sqrt{\xi_2}, \quad \sin\theta = \sqrt{1 - \xi_2},$$

после чего локальный вектор преобразуется базисом $(\boldsymbol{u}, \boldsymbol{v}, \boldsymbol{n})$.

8.2 Metal: GGX visible-normal sampling

Для metal shader использует $rough = 1 - smoothness$, $\alpha = rough^2$:

$$D(h) = \frac{\alpha^2}{\pi((1 - n_h^2) + n_h^2\alpha^2)^2},$$

$$G_1(c, \alpha) = \frac{2c}{c + \sqrt{\alpha^2 + (1 - \alpha^2)c^2}}, \quad G = G_1(n_v, \alpha)G_1(n_l, \alpha),$$

$$F = F_0 + (1 - F_0)(1 - v_h)^5, \quad f = F\frac{DG}{4n_v n_l}, \quad p_h = \frac{DG_1(n_v, \alpha)}{4n_v}.$$

Вес нового направления:

$$\boldsymbol{\beta} \leftarrow \boldsymbol{\beta} \odot f\,\frac{n_l}{p_h}.$$

Сэмплирование выбирает видимую микронормаль через stretched view vector и disk-параметризацию; это не простое равномерное сэмплирование GGX.

8.3 Mirror и гладкий metal

$$\boldsymbol{\omega}_r = \boldsymbol{d} - 2(\boldsymbol{d}\cdot\boldsymbol{n})\boldsymbol{n}, \qquad \boldsymbol{\beta} \leftarrow \boldsymbol{\beta} \odot \boldsymbol{albedo}\, reflectivity.$$

Для metal дополнительно применяется цветной F из предыдущей формулы. Mirror не получает direct-light sample и отражает идеально.

8.4 Dielectric: интерфейс сред

Текущая среда определяется не только front/back флагом: shader ищет ближайший dielectric object, содержащий точку после ULP-offset. Поэтому вложенные стеклянные объекты имеют стекоподобную семантику.

$$\eta = \frac{\eta_1}{\eta_2}, \quad c = \max(-\boldsymbol{d}\cdot\boldsymbol{n}, 0), \quad s_t^2 = \eta^2(1-c^2).$$

При $s_t^2 > 1$ возникает TIR и используется reflection. Иначе:

$$\boldsymbol{r}_\perp = \eta(\boldsymbol{d} + c\boldsymbol{n}), \qquad \boldsymbol{r}_\parallel = -\sqrt{1 - \|\boldsymbol{r}_\perp\|^2}\boldsymbol{n}, \quad \boldsymbol{d}_t = \boldsymbol{r}_\perp + \boldsymbol{r}_\parallel.$$

Вероятность отражения — точные уравнения Френеля:

$$s_t^2 = \eta^2(1-c^2), \quad c_t = \sqrt{1-s_t^2},$$

$$R = \frac{1}{2}\left[\left(\frac{\eta c - c_t}{\eta c + c_t}\right)^2 + \left(\frac{c - \eta c_t}{c + \eta c_t}\right)^2\right].$$

При преломлении $\boldsymbol{\beta} \leftarrow \eta^2\boldsymbol{\beta}$ и обновляется $etaScale \leftarrow etaScale/\eta^2$; при отражении вес равен $(1,1,1)$. На пути внутри dielectric прозрачность дополнительно умножается на

$$T_{glass}(d) = \exp(-\boldsymbol{\sigma}\, scale\, d) \odot \boldsymbol{tint}^{\,d}.$$

9 Прямое освещение и MIS

9.1 Список света

Источники выбираются из emissive sphere и box. За одну вершину shader samples $K = \min(|\mathcal{L}|, 8)$ источников; если источников больше восьми, стартовый индекс случайный и берётся циклическое окно. Inclusion factor:

$$q = \min\left(1, \frac{8}{|\mathcal{L}|}\right).$$

Для spherical light точка выбирается по видимому телесному углу. В терминах центра на расстоянии d и радиуса r:

$$s^2 = \frac{r^2}{d^2}, \quad 1 - \cos\theta = \frac{s^2}{1 + \sqrt{1-s^2}}, \quad p_\Omega = \frac{1}{2\pi(1-\cos\theta)}.$$

Для box видимые грани имеют площади

$$A_x = s_y s_z, \quad A_y = s_x s_z, \quad A_z = s_x s_y;$$

грань обнуляется, если reference находится внутри её slab. После выбора точки равномерно по суммарной площади A:

$$p_\Omega = \frac{r^2}{A\,|\boldsymbol{n}_L \cdot (-\boldsymbol{\omega})|}.$$

Для shadow ray проверяется occlusion всеми объектами, кроме самого emitter; затем учитывается glass/fog transmittance. Поверхностный direct contribution:

$$\Delta L = \boldsymbol{f} \odot \boldsymbol{L}_e \odot T\, \frac{n_l}{p_L q + [\text{continuation}] p_B}.$$

При первом/последнем bounce p_B не добавляется, иначе denominator реализует MIS balance heuristic между light sampling и BSDF sampling.

При прямом попадании случайного пути в источник его вклад умножается на

$$w_e = \frac{p_{prev}}{p_{prev} + p_{LQ}},$$

а если предыдущий PDF нулевой, $w_e = 1$.

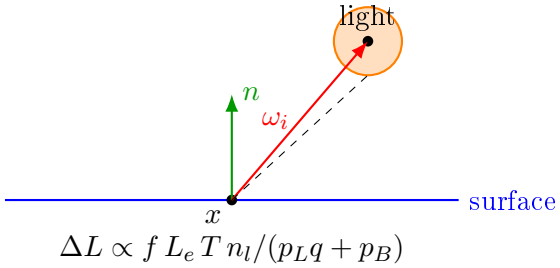


Figure 3: Next Event Estimation: отдельный shadow ray к сэмплу источника.

10 Итеративный path loop

Для каждого bounce $b = 0, \ldots, D-1$ shader выполняет следующие действия:

1. Проверяет $\max(\beta_r, \beta_g, \beta_b) > 0$.
2. Ищет ближайшее пересечение до $R_{\max}$ и получает расстояние d.
3. Если луч ещё не внутри стекла и включён volumetric fog, sample-ит возможное рассеяние до ближайшей поверхности/неба. При событии изменяет throughput, добавляет direct volume light, сэмплирует фазу и продолжает путь из точки объёма.
4. Иначе умножает throughput на стеклянное затухание текущей среды.
5. При miss добавляет $\boldsymbol{\beta} \odot \mathit{background}(\boldsymbol{d})$ и завершает путь. Для legacy non-volumetric fog с affect-sky фон дополнительно проходит $F(L, d) = Le^{-\mathit{density}\, d} + C(1 - e^{-\mathit{density}\, d})$ при $d = 50$.
6. При emissive hit добавляет $\boldsymbol{\beta} \odot \mathit{emit} \cdot w_e$ только для front face и завершает путь.
7. Для Lambert/metal/mirror/dielectric вычисляет direct/BSDF/refraction, сохраняет p_{prev} и previous point, умножает throughput и строит offset ray.
8. После $b \geq 3$ запускает Russian roulette:

$$p_{rr} = \min(1, \mathit{etaScale} \max(\beta_r, \beta_g, \beta_b)).$$

При отказе путь завершается, при продолжении $\boldsymbol{\beta} \leftarrow \boldsymbol{\beta}/p_{rr}$.

11 Объёмный туман

Если явные коэффициенты не заданы, из legacy density d и scatter q строится:

$$\sigma_s = dq, \qquad \sigma_a = \max(d - \sigma_s, 0), \qquad \sigma_t = \sigma_s + \sigma_a.$$

Для heterogeneous fog коэффициенты умножаются на

$$m(\boldsymbol{x}) = (1 - k) + 2kN(\boldsymbol{x}),$$

где N — hash/fBm noise с 1–5 октавами.

GPU выбирает majorant:

$$\mu = \big(\max(\sigma_s, 0) + \max(\sigma_a, 0)\big)(1 + k),$$

с fallback на density, если физические коэффициенты нулевые. Кандидат дистанции delta tracking:

$$t \leftarrow t - \frac{\ln(1 - \xi)}{\mu}.$$

Если $t \geq d$, событие объёма не произошло. Иначе принимается с вероятностью $\sigma_t(\boldsymbol{x})/\mu$. Вес принятого scattering event:

$$\boldsymbol{w}_{fog} = \mathrm{clamp}(fogColor, 0, 1)\frac{\sigma_s(\boldsymbol{x})}{\sigma_t(\boldsymbol{x})}.$$

Для неоднородной shadow transmittance применяется ratio tracking:

$$T \leftarrow T\left(1 - \frac{\sigma_t(\boldsymbol{x})}{\mu}\right)$$

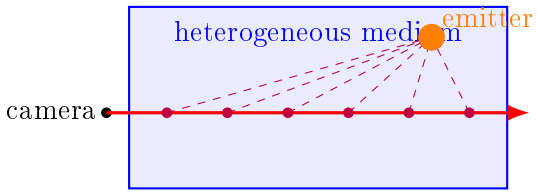
на каждой случайной позиции до конца отрезка. В однородном случае shader использует аналитическое $T = \exp(-\mu d)$.

Фаза Хеньи–Грина:

$$p_{HG}(\cos\theta) = \frac{1 - g^2}{4\pi(1 + g^2 - 2g\cos\theta)^{3/2}}.$$

Volume NEE использует ту же схему максимум восьми источников, а contribution имеет вид

$$\Delta L_{vol} = L_e \frac{p_{HG}}{p_{LQ} + [\text{continuation}]p_{HG}} T_{shadow}.$$



$t \sim \mathrm{Exp}(\mu)$, accept with σ_t/μ, weight σ_s/σ_t

Figure 4: Delta tracking вдоль луча и прямое освещение точки рассеяния.

+

12 Накопление, HDR и постобработка

После B_i путей одного pass для пикселя:

$$A_{x,y} \leftarrow A_{x,y} + \sum_{j=0}^{B_i - 1} L_{x,y,k+j}, \qquad \overline{L}_{x,y} = \frac{A_{x,y}}{N_i}.$$

Именно N_i передаётся как `uSampleCount`; поэтому последний неполный batch также усредняется корректно.

12.1 Тонемаппинг и 8-bit output

Для каждой положительной HDR-компоненты:

$$A(x) = \mathrm{clamp}\left(\frac{x(2.51x + 0.03)}{x(2.43x + 0.59) + 0.14}, 0, 1\right), \qquad C_8 = 255\sqrt{A(x)} + dither.$$

NaN, infinity и отрицательные значения становятся нулём. Альфа всегда равна 255. Это ACES-like mapping плюс square-root gamma; это не linear-to-sRGB transfer function.

12.2 CPU denoise и smooth

При включённом denoise и $S \geq 4$ после readback запускаются три прохода bilateral фильтра на радиусах $1, 2, 4$. Для соседнего пикселя j:

$$w_{sp} = \exp\left(-\frac{\Delta x^2 + \Delta y^2}{2\sigma_s^2}\right), \quad b = \frac{1}{2\max(\sigma_r^2\, 255^2, 10^{-4})},$$

$$w = \frac{w_{sp}}{(1 + (\Delta R^2 + \Delta G^2 + \Delta B^2)b)^2}, \qquad C' = \frac{\sum_j wC_j}{\sum_j w}.$$

Дополнительный smooth выключен по умолчанию; если включён, его результат смешивается с исходным byte-color с strength и радиусом 1–5.

13 Фактические настройки исполнения

Параметр	Реальное значение/правило
Normal work group	32×32
Wireframe work group	16×16
Samples per pass	default 4, clamp $1 \ldots 16$
Dispatch tile	старт 64, предел по умолчанию 256, диапазон 32–512; адаптируется по времени fence
Max light samples	8
Max smooth radius	5
GPU texture	RGBA32F
Accumulation	SSBO: $3WH$ float32
Readback	два PBO, RGBA float32
Temporal flag	target texture ping-pong есть, но shader не вызывает texture lookup предыдущего кадра
Denoise	CPU, по умолчанию включён для GPU при $S \geq 4$
GPU fallback	при ошибке GPU service запускает CPU renderer

14 Проверка соответствия коду

Ниже перечислены точки, которые следует проверять при изменении математики:

Математический блок	Источник истины в проекте
Backend/fallback	`internal/usecases/render/service.go`: `IntoWithCancel`, `renderIntoGPU`
Packing/stride	`internal/transport/gpu/packing.go`: `packMaterials`, `packObjects`, `packFogBlock`
GL resources/readback	`internal/transport/gpu/setup.go`, `gpu.go`: `resizeGPUResources`, `tonemapHDRToRGBA`
Dispatch/passes	`internal/transport/gpu/samples.go`: `samplePasses`, `dispatchTiles`, `dispatchGroups`
Camera/RNG/path	`internal/transport/gpu/shaders.go`, `physics_shader.go`, `transport_shader.go`
Lights/BSDF/MIS	`lighting_shader.go`, `physics_shader.go`
Fog	`fog_shader.go`, `physics_shader.go`
CPU parity	`internal/usecases/render/cpu_gpu_parity.go`, `physics.go`, `fog_physics.go`
Validation	`scenes/validation/README.md` и GPU internal tests

Для регрессионной проверки применяются команды:

```
go test ./internal/transport/gpu ./internal/usecases/render ./internal/service
go run ./cmd/validate-render -scene scenes/validation/glass_grid.json \
  -backend gpu -width 320 -height 180 -spp 64 -seed 1 -out bin/glass.png
PATHTRACER_GPU_INTEGRATION=1 go test ./internal/transport/gpu \
  -run '^TestGPU(PhysicsAudit|Integration)$' -count=1 -v -timeout=180s
```

В среде без аппаратного GL последний контур может быть недоступен; намеренное использование software Mesa требует `PATHTRACER\_GPU\_ALLOW\_SOFTWARE=1`. MSE/PSNR после ACES и gamma измеряют качество изображения, но не доказывают точность линейного интеграла.

15 Итоговая математическая схема

Для пикселя и sample k фактическая оценка имеет вид:

$$L_k = \mathrm{PathTrace}\left(\mathrm{CameraRay}(x, y, \xi_k), \mathcal{O}, \mathcal{M}, \mathcal{L}, \mathcal{F}, D, R_{\max}\right),$$

где внутри каждого bounce:

$$\begin{aligned}
&\text{hit/medium event}\\
&\beta \leftarrow \beta \odot T_{glass/fog}\\
&L \leftarrow L + \beta \odot (L_e + L_{direct})\\
&\omega_{next} \sim p_{Lambert/GGX/Fresnel/HG}\\
&\beta \leftarrow \beta \odot \frac{f\,|n \cdot \omega_{next}|}{p} \quad \text{(где применимо)}\\
&\text{Russian roulette после третьего bounce.}
\end{aligned}$$

Затем GPU выполняет:

$$A \leftarrow A + \sum_k L_k, \qquad \overline{L} = \frac{A}{N}, \qquad C_8 = 255\sqrt{\mathrm{ACES}(\overline{L})} + dither.$$

Именно последняя формула, а не clamp/Firefly reduction, является текущей границей между линейным HDR накопителем и отображаемым изображением.